Lua scripting

Open Factory Studio runs Lua scripts for the Project > New templates, for the Tools menu and from Tools > Script editor. The functions are the same as in Fighter Factory Studio, so scripts written for it work here too. Where something is different, this page says so.

Running scripts

Kind	Folder	How it runs
Common	data/scripts/common, data/engines/<engine>/scripts/common	Run by the program itself (the Throw creator runs engines/mugen/scripts/common/Throw.lua).
Project	data/scripts/project/<Category>/<Name>.lua, data/engines/<engine>/scripts/project/...	Project > New. The script returns the new project, or nil to cancel. A <Name>.png next to it is used as its icon.
Code snippet	.../scripts/snippet	Listed in the Script editor (Fighter Factory Studio doesn't use them either).
Tool	data/scripts/tool, data/engines/<engine>/scripts/tool	One Tools menu item per file, named after the file. The script runs, then its main().

Every run starts with a fresh Lua state. The script runs from top to bottom, and if it has a global function called main, that is called after it. In the Script editor, the text in the editor runs, and messages name it after its first line. ScriptPath is only set for scripts run from a file.

Errors look the same as in Fighter Factory Studio, with a traceback:

```
sol: runtime error: [string "-- my script"]:4: attempt to index a nil value (local 't')
stack traceback:
        [string "-- my script"]:4: in main chunk
```

Syntax errors start with "sol: syntax error:". Errors inside main() are shown too. A long running script doesn't freeze the window, and Run (or Esc) in the Script editor stops it.

Globals

Name	What it is
FF	The API table (OFS works too).
MUGEN	The engine's table with its enumerations. Every engine adds one under its own name.
ScriptPath	The running script's folder, without a slash at the end (nil for text from the editor).
debugOutput(value)	Writes its first value (a string or a number) to the Script editor's output, as it is.
print(...)	Writes its values, separated by tabs, and a line break to the output. Fighter Factory Studio ignores print().

Name	What it is
base	The same table as _G.

The libraries you can use are base (with dofile, loadfile and load), string, table, math and utf8. io, os, debug, package and coroutine aren't there.

How things work

- Properties are fields: page.title = "...", x = sprite.width.
- Methods are called with a colon: wizard:addPage(page). Calling one with a dot is an error.
- Enumerations are read-only tables of numbers. Flags combine with |.
- FF.projectAt and the at() methods of sprites, palettes, animations, frames, boxes and sounds count from 0. Lists returned as Lua tables count from 1.
- Wrong arguments give a Lua error. Nothing crashes the program.

FF functions

Function	Returns
FF.wizard([parent])	A new Wizard (owned by the main window if there's no parent).
FF.wizardPage([parent])	A new WizardPage.
FF.radioButton(text, parent)	A new RadioButton inside the widget parent.
FF.checkBox(text, parent)	A new CheckBox.
FF.comboBox(parent)	A new ComboBox.
FF.verticalLayout([parent]), FF.horizontalLayout([parent]), FF.gridLayout([parent])	A new layout, set on parent if you give one.
FF.label(text, parent), FF.lineEdit(parent), FF.spinBox(parent)	A Label, LineEdit or SpinBox. Not in Fighter Factory Studio.
FF.messageBox(text[, title])	Shows a message. Not in Fighter Factory Studio.
FF.translate([text])	The text in the user's language (from the script's <name>_<language>.qm), or the text as it is. "" without an argument.
FF.emptyProject(engine, type)	A blank project for an engine ("mugen") and a type (MUGEN.ProjectType.Character and so on). Outside templates it opens right away. nil for an unknown engine.
FF.projectFromFile(fileName[, asCopy])	The project of a main file (relative names start from ScriptPath). With asCopy, an untitled copy that gets saved under a new name. Outside templates it opens right away.
FF.projectCount()	How many projects are open.
FF.projectAt(index)	The open project at index (from 0), or nil.
FF.currentProject()	The current project, or nil.

Function	Returns
FF.currentEngine()	The Engine of the current project, or nil.
FF.engineList()	A table of the engines by name: { mugen = Engine }.
FF.engineByName(name)	An Engine, or nil.

Enumerations

Enumeration	Values
FF.Align	Left 1, Right 2, HCenter 4, Top 32, Bottom 64, VCenter 128
FF.FileType	Main 0, Sprites 1, Sounds 2, Animations 3, Scripts 4
FF.Filter	Dirs 1, Files 2, AllEntries 7, NoSymLinks 8, AllDirs 1024, CaseSensitive 2048, NoFilter -1
FF.SortFlag	Name 0, Time 1, Size 2, DirsFirst 4, Reversed 8, IgnoreCase 16, DirsLast 32, Type 128, NoSort -1
FF.OpenFlags, FF.OpenMode	ReadOnly 1, WriteOnly 2, ReadWrite 3
FF.ShareMode	NoShare 0, Share 1, AskToShare 2
FF.Flip	None 0, Horizontal 1, Vertical 2, Both 3
FF.Mode	None 0, Addition 1, Subtraction 2, Overlay 3, HardLight 4, Dodge 5, Opacity 6
FF.PaletteMode	Ignore 0, OwnPalette 1, AdaptImage 2, SwapPalette 3
FF.Type	Attack 1, Collision 2
FF.MoveMode	MoveAnchor 0, KeepAnchor 1
FF.MoveOperation	NoMove 0, Start 1, StartOfLine 3, StartOfWord 5, PreviousLine 6, PreviousCharacter 7, PreviousWord 8, End 11, EndOfWord 14, EndOfLine 15, NextLine 16, NextCharacter 17, NextWord 18
FF.SelectionType	WordUnderCursor 0, LineUnderCursor 1, Document 3
MUGEN.ProjectType	Character 0, Stage 1, Storyboard 2, Motif 3, Font 4, Select 5, Fight 6

FF.Filters.new([value]) and FF.SortFlags.new([value]) give a flags value (a number), and FF.StringList.new(...) gives a list (a Lua table). They are for the arguments of Archive:entries.

Classes

FF.Engine

Properties: name, displayName (not in Fighter Factory Studio), path (the engine's data folder). Methods: singleFileFilters() (the file patterns of each FF.FileType), createFile() (not available).

FF.Project

Properties: name, fileName, path, type (a ProjectType value), version ("M.U.G.E.N 1.0"), engine, isModified, isEmpty, archive, parent. Methods: files(), filesByType(type), fileByType(type), saveAll().

FF.ParentProject

Properties: engine, archive. Methods: save(), saveAs(fileName) (Project > Save as: a copy of the whole project next to fileName, the open project stays as it is), backup() (Project > Create backup, returns the backup's path), restoreBackup(fileName) (Project > Load backup, the files are read again), openedProjects().

FF.ProjectFile

Properties: fileName, fileExists, isEmpty, isModified, isReadOnly, project, archive. Methods: asSourceCode(), asSprites(), asAnimations(), asSounds() (each one is nil for another kind of file), beginChange() and endChange() (everything in between is one undo step), undo(), redo(), reset() (reads the file again), toFile(path) (writes a copy), fromFile(path) (text files: takes the text of another file).

FF.CodeDocument (a ProjectFile)

Properties: title, text, lineCount, cursor (a CodeCursor at the end). Methods: fromString(text), insertCode(text[, position]), insertCodeBefore(line, text) and insertCodeAfter(line, text) (whole lines, from 1), insertCodeOnCursor(cursor, text).

FF.CodeCursor

Properties: position, atStart, atEnd, selectedCode. Methods: movePosition(operation[, mode[, count]]), select(type), clearSelection(), insertCode(text), removeSelectedCode().

FF.Sprites

Properties: count, palettes. Methods: at(index), find(group, number), remove(index), clear(). add and insert are not available.

FF.Sprite

Properties: index, isEmpty, group, number, x, y (the axis; these four can be changed), width, height, imageProperties (FF.ImageProperties: format 8 or 32, width, height), palette, parent. Methods: duplicate(). crop, setImage and splitRGB are not available.

FF.Palettes, FF.SpritePalette

Palettes: count, sprites, at(index), find(group, number). SpritePalette: index, group, number, parent, colorTable (a table of FF.RGB from 1), setColorTable(colors), and the PropertyItem methods.

FF.Animations

Properties: count. Methods: at(index), find(number), indexOf(animation), remove(index). add, insert and duplicate are not available.

FF.AnimationCode

Properties: number, displayName ("0 - Standing"), text (its comment line), code (its text), frameCount, frames. Methods: frameAt(index), frameIndex(frame), and the PropertyItem methods.

FF.AnimationFrameCode

Properties: group, number, offsetX, offsetY, offset ({x, y}), time, scaleX, scaleY, scale ({x, y}), angle, flip (an FF.Flip value), loopStart, blend (FF.AnimationFrameBlend: mode, source, dest). Methods: attackBoxes(), collisionBoxes(), and the PropertyItem methods.

FF.HitBoxes, FF.HitBoxCode

HitBoxes: boxType (1 attack or 2 collision, plus 4 for default boxes), count, at(index). HitBoxCode: boxType, code (its line), left, top, right, bottom, rect ({left, top, right, bottom}).

FF.Sounds, FF.Sound

Sounds: count, at(index), find(group, number). Sound: index, group, number, size. In Fighter Factory Studio, asSounds() returns an object you can't do anything with.

PropertyItem methods

propertyCount(), propertyName(index) and propertyValue(index) (from 0) give the rows of an editor's property panel. AnimationCode has DisplayName, Name and Identifier. AnimationFrameCode has Identifier, Offset, Delay, Scale, Angle, Flip, LoopPoint, BlendMode, InterpolateBlend, InterpolateOffset, InterpolateScale and InterpolateAngle. SpritePalette has Identifier, SubItem and ColorCount. The values can be read.

FF.Archive

FF.Archive.instance(fileName[, readOnly]) gives the archive of a ZIP file (read only), a folder, or the folder of a file. Properties: name, path. Methods: isReadOnly(), absoluteFilePath(name), absoluteFileName(name), fileExists(name), directoryExists(name), isWritable(name), entries(path, nameFilters, filters, sort) (a table of names), openFile(name, openFlags) (a File or nil), closeFile(file), removeFile(name), renameFile(name, newName), flush(), release(). Names are relative to the folder, but full paths work too.

FF.File

Properties: pos, size, atEnd, isOpen, isSequential. Methods: read([count]), readAll(), readLine(), write(data), seek(pos), reset(), close().

Values

FF.RGB.new(r, g, b) (r or red, g or green, b or blue), FF.RGBA.new(r, g, b, a) (also a or alpha, toRGB(), fromRGB(rgb)), FF.Margins.new(left, top, right, bottom).

Widgets

- FF.Widget: windowTitle, enabled, visible, toolTip.
- FF.Dialog: exec(). FF.Wizard (a Dialog): addPage(page), exec() (1 for Finish, 0 for Cancel).
- FF.WizardPage: title, subTitle.
- FF.RadioButton, FF.CheckBox: checked, text.
- FF.ComboBox: addItem(text[, data]), count, currentIndex, currentText, currentData.
- FF.Label: text. FF.LineEdit: text, placeholderText. FF.SpinBox: value, minimum, maximum.
- FF.Layout: margin, spacing, margins. FF.BoxLayout (and FF.VerticalLayout, FF.HorizontalLayout): addWidget, addLayout, addSpacing, addStretch, addVerticalSpacer,

addHorizontalSpacer. FF.GridLayout: addWidget(widget, row, column[, rows, columns]), addLayout(...), addVerticalSpacer(row, column), addHorizontalSpacer(row, column).

Classes you can't get

FF.CodeNode, FF.ParentCode, FF.GroupCode, FF.ValueCode, FF.NamedCode, FF.Organizable, FF.PropertyItem, FF.Progress, FF.Debugger, FF.DebuggerBase, FF.Emulator and FF.Variable exist like in Fighter Factory Studio, but no function returns one.

Translating scripts

Save for translation in the Script editor writes <name>.ts next to a saved script. It is a Qt Linguist file with every text passed to FF.translate as a literal, in a context named after the script. Translate it in Qt Linguist and release it as <name>_<language>.qm (for example Empty_fr.qm) next to the script. FF.translate then uses it when the program runs in that language.